SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CSA04 COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.* *(BL2, BL3)*

QUESTIONS: 1 **Total Marks:100**

Annexure A

Concept Mapping

Code of Conduct:

*I **P.Saketh Kumar Reddy(192525340)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Concept Mapping

Question 1
Show a multi-level concept map integrating:
Operating System Types → OS Structures → System Calls → OS Functions.
Your map must show cross-links (e.g., how OS structure influences system call implementation). **(BL2)**

Question 2
Construct a comprehensive concept map representing:
Process Management, Memory Management, File System, and I/O System.
Include interdependencies and indicate where system calls interact with each component. **(BL3)**

Question 3
Sketch a comparative concept map for:
Batch OS, Time-Sharing OS, Distributed OS, and Real-Time OS.
Highlight differences, similarities, advantages, and limitations using linking phrases. **(BL2)**

Question 4
Illustrate a dynamic concept map showing the execution flow:
User Program → API → System Call Interface → Kernel → Hardware → Response back to User.
Include feedback loops and control flow paths. **(BL3)**

Question 5
Sketch a hierarchical + functional concept map for system calls:
Process, File, Device, and Information system calls.
Also show how they relate to OS services like scheduling, memory allocation, and device handling. **(BL3)**

RUBRICS FOR CALCULATION

Concept Mapping					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

CSA04 / CO1 / MAP 1

Operating Systems: Types → Structures → System Calls → Functions

QUESTION 1 | The map follows the required four-level flow and adds cross-links explaining architectural trade-offs.

OPERATING SYSTEM TYPES

sets workload + timing needs

Batch OS
- Runs queued jobs with little interactive input.
- Optimizes throughput and job completion.

Time-sharing OS
- Pre-empts CPU time among interactive users/processes.
- Optimizes responsiveness and fair sharing.

Distributed OS
- Coordinates networked computers and shared resources.
- Depends on communication and remote-service support.

Real-time OS
- Responds within defined timing constraints.
- Prioritizes predictable latency and deadline handling.

OS STRUCTURES

organizes kernel services

Monolithic kernel
- Most core services run in kernel space.
- Direct internal dispatch is efficient, but the trusted core is larger.

Layered structure
- Services are arranged in defined layers.
- Separation is clearer; requests may cross multiple layer boundaries.

Microkernel
- Keeps the kernel minimal; many services run as user-space servers.
- IPC improves isolation/modularity but can add transitions.

Modular / hybrid
- Uses loadable modules and/or combines structural styles.
- Balances extensibility, performance and compatibility.

SYSTEM CALLS & INTERFACE

provides controlled entry points

How a system call works
- User program calls an API/library wrapper.
- A trap/syscall instruction enters kernel mode; the handler validates, dispatches and returns a value or error.

Main call families
- Process: create, execute, wait, terminate.
- File/device: open, read, write, close, ioctl.
- Information/communication: stat, getpid, socket (platform-dependent).

Useful examples
- fork(), exec(), wait(), exit() - process control.
- open(), read(), write(), close() - file access.
- mmap(), munmap(), ioctl(), socket() - memory/device/network operations.

OS FUNCTIONS

delivers protected services

Process management
- Creates, schedules, synchronizes and terminates processes/threads.
- Handles IPC and coordinates CPU use.

Memory management
- Allocates memory and manages virtual address spaces, paging and protection.
- Supports controlled sharing between processes.

File-system services
- Organizes names, directories, metadata and persistent data.
- Enforces file access permissions.

I/O, security & communication
- Coordinates drivers, interrupts, buffering and device access.
- Also supports protection, networking and error/accounting services.

CROSS-LINKS | How OS structure and workload affect system-call implementation

Monolithic → direct dispatch
- The system-call handler can reach in-kernel services with fewer extra transitions.
- Trade-off: a larger privileged code base.

Microkernel → IPC boundary
- A call may be routed to a user-space server using inter-process communication.
- Trade-off: extra messages/context switches; benefit: service isolation.

Layered → service path
- A request may pass through multiple layers before reaching a device or data service.
- Trade-off: clearer boundaries versus possible path overhead.

OS type → timing / communication
- Real-time systems need bounded response times; time-sharing emphasizes responsiveness.
- Distributed systems rely more heavily on communication and remote-service calls.

CORE IDEA
- Type = operating environment; structure = internal organization.

CONTROL / SAFETY
- System calls are the controlled boundary from user mode to protected kernel services.

EXAMPLE OF A CROSS-LINK
- open/read/write can involve the syscall interface, file system, memory buffers and I/O drivers.

Reading anchors: Silberschatz, Galvin & Gagne, Operating System Concepts; Tanenbaum & Bos, Modern Operating Systems. Content is a synthesis of standard OS concepts.

CSA04 / CO1 / MAP 2

Operating System Resource Management: Four Interdependent Components

QUESTION 2 | Process, memory, file and I/O management are separate responsibilities connected through kernel services and system calls.

SYSTEM CALL INTERFACE

User program / API → syscall trap → kernel dispatch → subsystem handler → return or error

1 PROCESS MANAGEMENT

- Purpose: control the life cycle of processes and threads.
- Core concepts: process states, PCB/context, CPU scheduling, synchronization, IPC, creation and termination.
- System calls: fork(), exec(), wait(), exit(), kill(), getpid() (names vary by OS).
- Interactions: each process has an address space; blocked processes may wait for I/O completion; file descriptors are associated with processes.

2 MEMORY MANAGEMENT

- Purpose: allocate, track and protect each process address space.
- Core concepts: virtual memory, paging/page tables, physical frames, page faults, sharing, memory protection.
- System calls: mmap(), munmap(), brk()/sbrk(), mprotect() where supported.
- Interactions: file-backed mappings connect files to memory; buffers, caches and DMA link memory with I/O.

process ↔ file descriptors

memory ↔ buffers / DMA

3 FILE SYSTEM

- Purpose: organize, name, secure and persist data on storage.
- Core concepts: files/directories, metadata, file descriptors, permissions, caching and consistency.
- System calls: open(), read(), write(), close(), lseek(), stat(), mkdir(), unlink().
- Interactions: filesystem requests are carried to storage through the I/O layer; mmap() can link file contents to virtual memory.

4 I/O SYSTEM

- Purpose: provide a safe, efficient interface to physical and virtual devices.
- Core concepts: device drivers, interrupts, DMA, queues, buffering, caching, spooling and device scheduling.
- System calls: read(), write(), ioctl(), poll()/select() and device-specific requests.
- Interactions: blocking requests put processes to sleep; completion interrupts/events let the kernel wake them; buffers coordinate transfers.

INTERDEPENDENCIES | Follow these links when explaining how the four components cooperate

PROCESS ↔ MEMORY
Each process owns a virtual address space; a page fault may block execution until memory is available.

PROCESS ↔ FILE SYSTEM
File descriptors are tracked for processes; permissions and ownership control access.

PROCESS ↔ I/O
A blocking I/O request can wait; completion wakes the process so scheduling can resume it.

MEMORY ↔ FILE SYSTEM
Page cache and memory-mapped files connect persistent file data with virtual memory.

MEMORY ↔ I/O
Buffers, caches and DMA coordinate data transfer between devices and RAM.

FILE SYSTEM ↔ I/O
File reads/writes are translated into storage requests handled by drivers and device queues.

KEY FLOW: User request → system-call interface → appropriate kernel subsystem → related subsystem/device work → status or data returned to the process.